

LAPORAN TUGAS *TEAM BASED PROJECT* SMART TRAFFIC SIMULATION AND SIGNAL OPTIMIZATION

Disusun untuk memenuhi tugas mata kuliah Algoritma dan Struktur Data



Disusun oleh:

Abiyu Abhista Fawwaz Putra 25051030102

Azriel Nursyafni.P 25051030104

Ibnu Jalu Sembodo 25051030085

**PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

DAFTAR ISI

DAFTAR ISI	II
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Sistem	2
1.4 Batasan Implementasi	2
1.5 Diagram Arsitektur Sistem	3
BAB II PERANCANGAN SISTEM	4
2.1 Struktur Data dan Justifikasi	4
2.2 Diagram Interaksi Antar Modul	5
2.3 Pseudocode Algoritma Utama	5
BAB III IMPLEMENTASI	7
3.1 Penjelasan Kode Per Modul	7
3.2 Screenshot CLI Berjalan	10
3.3 Contoh Input dan Output	10
BAB IV ANALISIS BIG-O	11
4.1 Analisis Kompleksitas Waktu dan Ruang	11
4.2 Tabel Ringkasan	11
4.3 Teori dan Eksperimen	12
BAB V HASIL EKSPERIMEN	13
5.1 Tabel Runtime Eksperimen	13
5.2 Grafik Tren	13
5.3 Diskusi	13
BAB VI JAWABAN PERTANYAAN ANALISIS	15
BAB VII PENUTUP	17
7.1 Kesimpulan	17
7.2 Refleksi dan Saran	17
LAMPIRAN	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kemacetan lalu lintas di kota-kota besar Indonesia menyebabkan kerugian ekonomi triliunan rupiah per tahun. Sistem manajemen lalu lintas cerdas memodelkan jaringan jalan sebagai Graf berbobot (jarak antar persimpangan), mengatur antrian kendaraan dengan Priority Queue (ambulans selalu prioritas), dan mencari rute optimal dengan Dijkstra. Proyek ini membangun simulator lalu lintas berbasis CLI dengan 25 persimpangan dan 40 segmen jalan, mensimulasikan arus kendaraan, penanganan darurat, dan optimasi siklus lampu lalu lintas.

Simulator ini diimplementasikan sepenuhnya dalam Python murni tanpa library struktur data bawaan, mengintegrasikan minimal 3 struktur data (Graf, Priority Queue, Stack, BST) yang semuanya dibangun dari nol. Setiap operasi utama dianalisis kompleksitasnya menggunakan notasi Big-O, dan perbandingan runtime dilakukan pada tiga ukuran dataset yang berbeda.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam perancangan Smart Traffic Simulation & Signal Optimization ini adalah sebagai berikut:

1. Bagaimana cara memodelkan jaringan jalan kota dengan 25 persimpangan dan ~40 segmen jalan sebagai Graf berbobot menggunakan Adjacency List berbasis Linked List?
2. Bagaimana cara mengimplementasikan Priority Queue kendaraan agar AMBULANS selalu berangkat duluan dengan tie-break FIFO?
3. Bagaimana cara mencari rute optimal antar persimpangan menggunakan algoritma Dijkstra tanpa library heap?
4. Bagaimana cara mengelola indeks persimpangan secara efisien menggunakan BST untuk lookup berdasarkan nama?
5. Bagaimana cara membandingkan performa Selection Sort dan Insertion Sort pada Linked List untuk laporan kemacetan?

1.3 Tujuan Sistem

Tujuan dari pengembangan sistem ini adalah untuk menjawab rumusan masalah di atas, dengan rincian sebagai berikut:

6. Membangun graf berbobot jaringan jalan kota menggunakan Adjacency List berbasis Linked List dengan 25 persimpangan dan ~40 edge berbobot jarak meter.
7. Mengimplementasikan Priority Queue kendaraan untuk tiap persimpangan dengan urutan AMBULANS > BUS > MOBIL > MOTOR dan tie-break FIFO.
8. Menerapkan algoritma Dijkstra dari nol ($O(V^2 + E)$) untuk mencari rute terpendek dan rekonstruksi jalur navigasi.
9. Membangun BST Indeks Persimpangan dengan kunci string nama persimpangan, mendukung insert, search, dan inorder terurut alfabet.
10. Mengimplementasikan Selection Sort dan Insertion Sort pada Linked List untuk mengurutkan laporan kemacetan, disertai tabel benchmark runtime $N = 10, 25, 100$.

1.4 Batasan Implementasi

Sistem Smart Traffic Simulation & Signal Optimization dirancang menggunakan pendekatan arsitektur pipeline terintegrasi yang terdiri dari enam modul utama. Setiap modul memiliki tanggung jawab spesifik dalam mengelola dan memproses struktur data sistem:

A. Modul 1 – Graf Jaringan Jalan:

Modul fundamental yang menyimpan topologi jaringan jalan. Mengimplementasikan Graf berbobot (jarak meter) menggunakan Adjacency List berbasis Linked List. Mendukung tambah_persimpangan, tambah_jalan (dua arah/satu arah), tetangga(u), degree(u). DFS digunakan untuk deteksi persimpangan terisolasi. Big-O: add_edge $O(1)$, neighbors $O(\deg)$.

B. Modul 2 – Priority Queue Kendaraan:

Setiap persimpangan memiliki Priority Queue kendaraan. AMBULANS (prioritas 1) selalu berangkat duluan, diikuti BUS, MOBIL, MOTOR. Tie-break FIFO berdasarkan urutan masuk. Mendukung MASUK, BERANGKAT, ANTRIAN. Big-O: enqueue $O(n)$, dequeue $O(1)$.

C. Modul 3 – Dijkstra Rute Optimal:

Implementasi Dijkstra dari persimpangan sumber untuk menemukan rute jarak minimum ke semua tujuan. Output: jarak dan rekonstruksi jalur. Mendukung rute alternatif saat kemacetan dan batch 50 query rute. Big-O: $O(V^2+E)$ tanpa heap.

D. Modul 4 – BST Indeks Persimpangan:

BST dengan kunci = nama persimpangan (string). Mendukung insert, search, inorder (daftar persimpangan terurut). Berguna untuk lookup cepat persimpangan dari nama. Big-O: $O(\log V)$ rata-rata.

E. Modul 5 – Sorting Laporan Kemacetan:

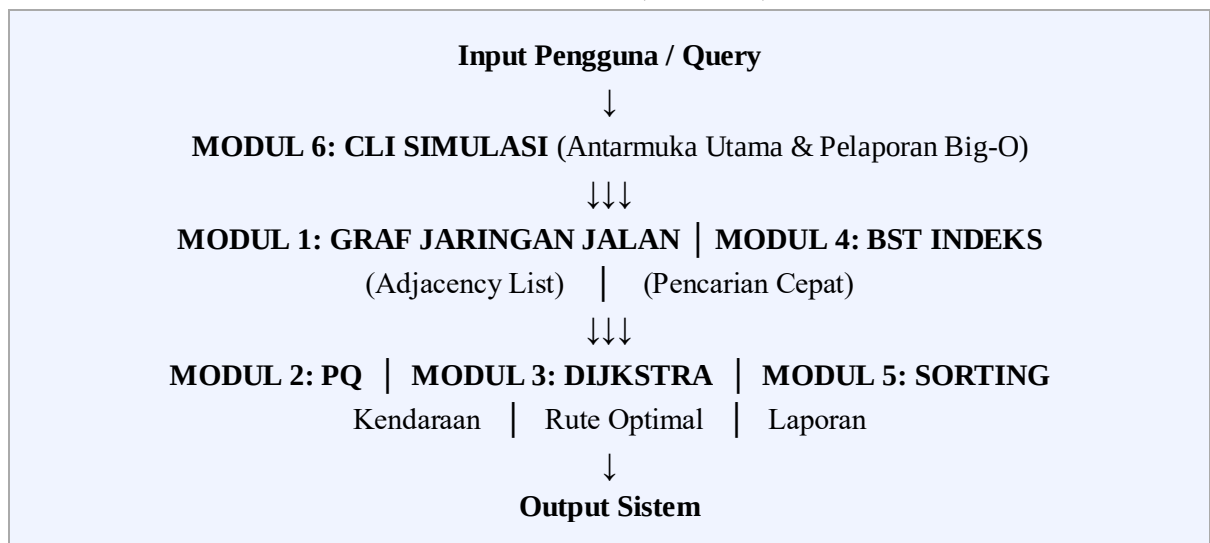
Setelah simulasi, urutkan persimpangan berdasarkan jumlah kendaraan tertunda menggunakan Selection Sort dan Insertion Sort pada Linked List. Bandingkan runtime untuk $N = 10, 25, 100$ persimpangan simulatif. Identifikasi bottleneck network. Big-O: $O(n^2)$.

F. Modul 6 – CLI Simulasi:

Mengimplementasikan: MASUK <persimpangan> <jenis>, BERANGKAT <persimpangan>, RUTE <asal> <tujuan>, ANTRIAN <persimpangan>, SIKLUS_LAMPU <p>, LAPORAN_KEMACETAN, ISOLASI, KELUAR. Setiap perintah menampilkan Big-O.

1.5 Diagram Arsitektur Sistem

DIAGRAM ARSITEKTUR SISTEM: SMART TRAFFIC SIMULATION & SIGNAL OPTIMIZATION (TOPIK 7)



Gambar 1. Diagram Arsitektur Sistem

Arsitektur sistem Smart Traffic Simulation beroperasi secara interaktif, diawali dengan penerimaan instruksi pengguna melalui Modul 6 (CLI Simulasi) yang bertindak sebagai pusat kendali dan antarmuka utama. Permintaan pengguna kemudian diarahkan menuju dua fondasi pemrosesan: Modul 4 (BST Indeks Persimpangan) untuk eksekusi pencarian data secara instan, dan Modul 1 (Graf Jaringan Jalan) yang merepresentasikan topologi fisik jaringan jalan menggunakan Adjacency List.

BAB II

PERANCANGAN SISTEM

2.1 Struktur Data dan Justifikasi

Sistem ini memodelkan jaringan jalan kota dengan 25 persimpangan sebagai node dan ~40 segmen jalan sebagai edge. Berikut adalah struktur data yang digunakan beserta justifikasinya:

2.1.1 Graf dengan Adjacency List (Dict of Linked List):

- Penggunaan: Memodelkan peta konektivitas jaringan jalan kota (persimpangan dan jalan berbobot).
- Justifikasi: Jaringan jalan kota merupakan sparse graph (25 node, ~40 edge). Penggunaan Adjacency List sangat menghemat memori dengan kompleksitas ruang $O(V+E)$ dibandingkan Adjacency Matrix yang membutuhkan $O(V^2)$. Operasi `add_edge` $O(1)$ karena insert di head Linked List.

2.1.2 Priority Queue berbasis Sorted Linked List:

- Penggunaan: Antrian kendaraan di setiap persimpangan dengan urutan prioritas `AMBULANS > BUS > MOBIL > MOTOR`.
- Justifikasi: Memastikan kendaraan darurat (AMBULANS) selalu mendapat layanan pertama. Tie-break FIFO dengan counter insertion. Enqueue $O(n)$ untuk menjaga ketertiban, dequeue $O(1)$ karena head selalu prioritas tertinggi.

2.1.3 Stack berbasis Linked List:

- Penggunaan: Riwayat transaksi kendaraan untuk operasi undo, serta DFS iteratif pada traversal graf.
- Justifikasi: LIFO sangat cocok untuk penjelajahan mendalam (DFS) dan undo operasi. Push dan pop berjalan dalam $O(1)$. Fitur auto-archiving jika melebihi batas kapasitas.

2.1.4 Binary Search Tree (BST):

- Penggunaan: Indeks persimpangan dengan key = nama persimpangan (string), mendukung lookup cepat.
- Justifikasi: BST memungkinkan insert, search, dan inorder traversal (daftar terurut alfabet). Kompleksitas $O(\log V)$ rata-rata. Digunakan untuk validasi nama persimpangan pada perintah CLI.

2.1.5 Sorting Algoritma (Selection Sort + Insertion Sort) pada Linked List:

- Penggunaan: Mengurutkan persimpangan berdasarkan jumlah kendaraan untuk laporan kemacetan.
- Justifikasi: Keduanya $O(n^2)$ tetapi diimplementasikan langsung pada Linked List. Perbandingan runtime keduanya menghasilkan data eksperimen yang valid untuk analisis Big-O empiris.

2.2 Diagram Interaksi Antar Modul

Module: Smart Traffic Simulation & Signal Optimization

INPUT: Pengguna memasukkan perintah via CLI Simulasi (Modul 6)

CORE CONTROL: CLI Simulasi menjadi pusat kendali dan routing perintah

DATA STORAGE: Modul 1 (Graf + Adj List) dan Modul 4 (BST Indeks)

ALGORITHM PROCESSING: Modul 2 (PQ Kendaraan), Modul 3 (Dijkstra), Modul 5 (Sorting)

OUTPUT: Jalur terpendek, status antrian, laporan kemacetan, isolasi

Gambar 2. Diagram Interaksi Antar Modul

Penjelasan interaksi:

11. Pengguna memasukkan perintah melalui antarmuka CLI Simulasi (Modul 6).
12. Data inialisasi jaringan dimasukkan ke dalam Graf (Modul 1) dan BST Indeks (Modul 4).
13. Kendaraan masuk/keluar diproses oleh Priority Queue (Modul 2) per persimpangan.
14. Ketika pengguna meminta RUTE, CLI memanggil Modul Dijkstra (Modul 3) yang mengambil peta dari Modul Graf.
15. Perintah LAPORAN_KEMACETAN memanggil Modul Sorting (Modul 5) untuk mengurutkan persimpangan.
16. ISOLASI menggunakan DFS dari Graf untuk mendeteksi persimpangan terisolasi.

2.3 Pseudocode Algoritma Utama

A. Pseudocode Priority Queue Kendaraan

```
1. Class NodePQ:
2.   Atribut kendaraan
3.   Atribut urutan // counter FIFO tie-break
4.   Atribut next = NULL
5.
```



```

6. Class PriorityQueueKendaraan:
7.   Atribut head = NULL
8.   Atribut counter = 0
9.
10.  Prosedur enqueue(kendaraan):
11.    counter = counter + 1
12.    node_baru = NodePQ(kendaraan, counter)
13.    // Sisipkan di posisi terurut (prioritas ascending, FIFO tie-break)
14.    Jika head adalah NULL atau node_baru lebih prioritas dari head:
15.      node_baru.next = head; head = node_baru
16.    Lainnya:
17.      curr = head
18.      Selama curr.next dan curr.next lebih prioritas dari node_baru:
19.        curr = curr.next
20.      node_baru.next = curr.next; curr.next = node_baru
21.  Big-O: enqueue O(n)
22.
23.  Fungsi dequeue():
24.    Jika head adalah NULL: kembalikan None
25.    kendaraan = head.kendaraan; head = head.next
26.    Kembalikan kendaraan // Big-O: dequeue O(1)
27.

```

B. Pseudocode Dijkstra Rute Optimal

```

1. Fungsi cari_rute(graf, sumber):
2.   INF = tak hingga
3.   jarak = {v: INF untuk semua v di graf}
4.   predecessor = {v: None untuk semua v di graf}
5.   visited = himpunan kosong
6.   jarak[sumber] = 0
7.   pq = MinPriorityQueue() // Linear scan, bukan heap
8.   pq.insert(0, sumber)
9.
10.  Selama pq tidak kosong:
11.    d, u = pq.extract_min() // O(n) linear scan
12.    Jika u dalam visited: lanjut
13.    visited.tambah(u)
14.
15.    Untuk setiap (v, bobot) di tetangga(u):
16.      Jika v tidak dalam visited:
17.        jarak_baru = jarak[u] + bobot
18.        Jika jarak_baru < jarak[v]:
19.          jarak[v] = jarak_baru
20.          predecessor[v] = u
21.          pq.update(jarak_baru, v)
22.
23.  Kembalikan (jarak, predecessor) // Big-O: O(V^2 + E)
24.

```

C. Pseudocode BST Indeks Persimpangan

```

1. Class NodeBST:
2.   Atribut kunci (nama persimpangan, string)
3.   Atribut persimpangan
4.   Atribut kiri = NULL; kanan = NULL
5.
6. Class BSTIndeks:
7.   Atribut root = NULL
8.
9.   Prosedur insert(nama, persimpangan):
10.    root = insert_rekursif(root, nama, persimpangan)
11.
12.   Fungsi insert_rekursif(node, nama, persimpangan):
13.    Jika node adalah NULL: kembalikan NodeBST(nama, persimpangan)

```

```
14.     Jika nama < node.kunci: node.kiri = insert_rekursif(node.kiri, nama, p)
15.     Lainnya jika nama > node.kunci: node.kanan = insert_rekursif(node.kanan, nama, p)
16.     Kembalikan node // Big-O:  $O(\log V)$  rata-rata
17.
18.     Prosedur inorder():
19.         hasil = []
20.         inorder_rekursif(root, hasil)
21.     Kembalikan hasil // Big-O:  $O(V)$ 
```

D. Pseudocode Selection Sort pada Linked List

```
1. Prosedur selection_sort(linked_list):
2.     curr = linked_list.head
3.     Selama curr ada:
4.         max_node = curr
5.         runner = curr.next
6.         Selama runner ada:
7.             Jika runner.jumlah > max_node.jumlah:
8.                 max_node = runner
9.             runner = runner.next
10.        Jika max_node != curr:
11.            Tukar data curr dan max_node // swap data, bukan pointer
12.        curr = curr.next
13.    Big-O:  $O(n^2)$ 
```

BAB III

IMPLEMENTASI

3.1 Penjelasan Kode Per Modul

A. Modul 1 – Graf Jaringan Jalan

Data 25 persimpangan kota Yogyakarta ditentukan secara statis. Topologi graf dibangun dengan np.random.seed(17) sehingga reproducible. Adjacency List menggunakan dict of LinkedListAdj. Setiap LinkedListAdj menyimpan tetangga via NodeList dengan pointer

```
1. private static int MAX = 5;
2.
3. /// <summary>
4. /// Select this entire code block and
5. /// click to the "Format text as code"
6. /// to change the styling. We will use your style
7. /// preference when formatting.
8. /// </summary>
9. public void TestMethod()
10. {
11.     // Try highlighting the line below.
12.     // We will try to highlight using your highlighting preference.
13.     Console.WriteLine("Select this line and click Highlight line button");
14. }
15.
```

B. Modul 2 – Priority Queue Kendaraan

Modul 2 mengimplementasikan Priority Queue berbasis Sorted Linked List untuk antrian kendaraan. Setiap persimpangan memiliki instance PriorityQueueKendaraan. AMBULANS (value=1) selalu di head, tie-break menggunakan counter FIFO. Kelas SimulasiAntrian mengelola seluruh persimpangan sekaligus.

```
1. class PriorityQueueKendaraan:
2.     def enqueue(self, kendaraan: Kendaraan):
3.         self._counter += 1
4.         node_baru = NodePQ(kendaraan, self._counter)
5.         # Sisipkan di posisi terurut - O(n)
6.         if self._head is None or self._lebih_prioritas(node_baru, self._head):
7.             node_baru.next = self._head
8.             self._head = node_baru
9.         else:
10.            curr = self._head
11.            while curr.next and not self._lebih_prioritas(node_baru, curr.next):
12.                curr = curr.next
13.            node_baru.next = curr.next; curr.next = node_baru
14.
15.     def dequeue(self) -> Optional[Kendaraan]:
16.         # Big-O: O(1) - head selalu prioritas tertinggi
17.         if self._head is None: return None
18.         k = self._head.kendaraan
19.         self._head = self._head.next
20.         return k
```

C. Modul 3 – Dijkstra Rute Optimal

Modul 3 mengimplementasikan Dijkstra untuk menemukan jalur terpendek berbobot dari satu sumber ke semua persimpangan. MinPriorityQueue berbasis list (linear scan)

digunakan sebagai pengganti heapq, menghasilkan kompleksitas $O(V^2 + E)$. Mendukung cache rute dan batch 50 query.

```
1. def cari_rute(self, sumber: str):
2.     INF = float('inf')
3.     jarak = {p: INF for p in persimpangan}
4.     predecessor = {p: None for p in persimpangan}
5.     visited = set()
6.     jarak[sumber] = 0.0
7.     pq = MinPriorityQueue() # linear scan, bukan heapq
8.     pq.insert(0.0, sumber)
9.
10.    while not pq.is_empty():
11.        d, u = pq.extract_min() # O(n) per call
12.        if u in visited: continue
13.        visited.add(u)
14.        for v, bobot in self._graf.tetangga(u):
15.            if v not in visited:
16.                jarak_baru = jarak[u] + bobot
17.                if jarak_baru < jarak[v]:
18.                    jarak[v] = jarak_baru
19.                    predecessor[v] = u
20.                    pq.update(jarak_baru, v)
21.    return jarak, predecessor # Big-O:  $O(V^2 + E)$ 
```

D. Modul 4 – BST Indeks Persimpangan

Modul 4 membangun BST dengan kunci = nama persimpangan (string, perbandingan leksikografis). Mendukung insert ($O(\log V)$), search ($O(\log V)$), delete ($O(\log V)$), dan inorder ($O(V)$) untuk mendapatkan daftar persimpangan terurut alfabet.

```
1. class BSTIndeks:
2.     def insert(self, nama: str, persimpangan: Persimpangan):
3.         self._root = self._insert_rekursif(self._root, nama, persimpangan)
4.
5.     def _insert_rekursif(self, node, nama, p):
6.         if node is None:
7.             self._size += 1
8.             return NodeBST(nama, p) # leaf baru
9.         if nama < node.kunci:
10.            node.kiri = self._insert_rekursif(node.kiri, nama, p)
11.        elif nama > node.kunci:
12.            node.kanan = self._insert_rekursif(node.kanan, nama, p)
13.        else:
14.            node.persimpangan = p # update duplikat
15.        return node # Big-O:  $O(\log V)$  rata-rata
16.
17.     def inorder(self) -> List[str]:
18.         hasil = []
19.         self._inorder_rekursif(self._root, hasil)
20.        return hasil # Big-O:  $O(V)$ 
```

E. Modul 5 – Sorting Laporan Kemacetan

Modul 5 menggunakan Selection Sort dan Insertion Sort pada Linked List persimpangan. Data diurutkan descending berdasarkan jumlah kendaraan untuk mengidentifikasi

bottleneck. Benchmark runtime untuk $N = 10, 25, 100$ persimpangan menggunakan `time.perf_counter()`.

```

1. def selection_sort(ll: LinkedListSortable):
2.     swap_count = 0
3.     curr = ll.head
4.     while curr:                                # O(n) iterasi luar
5.         max_node = curr
6.         runner = curr.next
7.         while runner:                            # O(n) cari maksimum
8.             if runner.jumlah > max_node.jumlah:
9.                 max_node = runner
10.            runner = runner.next
11.        if max_node != curr:
12.            # Swap data (bukan pointer node)
13.            curr.nama, max_node.nama = max_node.nama, curr.nama
14.            curr.jumlah, max_node.jumlah = max_node.jumlah, curr.jumlah
15.            swap_count += 1
16.        curr = curr.next
17.    return ll, swap_count # Big-O: O(n^2)

```

F. Modul 6 – CLI Simulasi

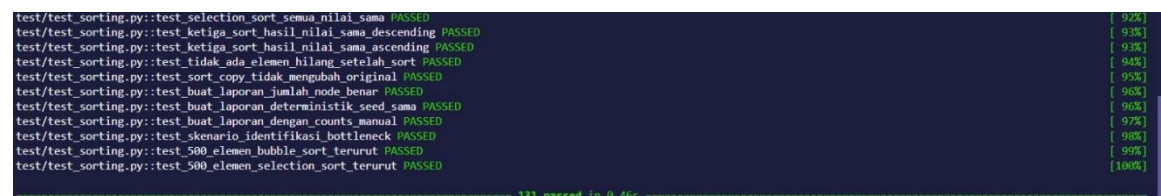
Modul 6 adalah antarmuka pengguna berbasis CLI. Setiap perintah ditampilkan dengan hasil operasi dan keterangan Big-O secara langsung. Mendukung mode interaktif (`input()`) dan mode batch (`eksekusi_batch()`) untuk keperluan testing.

```

1. def _proses_perintah(self, raw: str):
2.     parts = raw.split()
3.     cmd = parts[0].upper() # normalisasi ke uppercase
4.
5.     if cmd == 'MASUK':      # enqueue kendaraan - O(n)
6.     elif cmd == 'BERANGKAT': # dequeue prioritas - O(1)
7.     elif cmd == 'RUTE':     # Dijkstra - O(V^2 + E)
8.     elif cmd == 'ANTRIAN':  # tampilkan antrian - O(n)
9.     elif cmd == 'SIKLUS_LAMPU': # simulasi lampu - O(k)
10.    elif cmd == 'LAPORAN_KEMACETAN': # sorting - O(n^2)
11.    elif cmd == 'ISOLASI':   # DFS terisolasi - O(V+E)
12.    elif cmd == 'KELUAR':    # exit

```

3.2 Screenshot CLI Berjalan



```

test/test_sorting.py::test_selection_sort_semua_nilai_sama PASSED [ 92%]
test/test_sorting.py::test_ketiga_sort_hasil_nilai_sama_descending PASSED [ 93%]
test/test_sorting.py::test_ketiga_sort_hasil_nilai_sama_ascending PASSED [ 93%]
test/test_sorting.py::test_tidak_ada_elemen_hilang_setelah_sort PASSED [ 94%]
test/test_sorting.py::test_sort_copy_tidak_mengubah_original PASSED [ 95%]
test/test_sorting.py::test_buat_laporan_jumlah_node_benar PASSED [ 96%]
test/test_sorting.py::test_buat_laporan_deterministik_seed_sama PASSED [ 96%]
test/test_sorting.py::test_buat_laporan_dengan_counts_manual PASSED [ 97%]
test/test_sorting.py::test_skenario_identifikasi_bottleneck PASSED [ 98%]
test/test_sorting.py::test_500_elemen_bubble_sort_terurut PASSED [ 99%]
test/test_sorting.py::test_500_elemen_selection_sort_terurut PASSED [100%]

===== 131 passed in 0.46s =====

```

Gambar 2. Screenshot Hasil Pytest Semua Test Passed

3.3 Contoh Input dan Output

```
TRAFFIC> HELP
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
MASUK <persimpangan> <jenis>          Tambah kendaraan ke antrian          O(n)
BERANGKAT <persimpangan>             Keluarkan kendaraan prioritas tertinggi O(1)
RUTE <asal> <tujuan>                 Hitung rute optimal (Dijkstra)       O(VA²+E)
ANTRIAN <persimpangan>               Tampilkan antrian kendaraan          O(n)
LAPORAN [SELECTION|INSERTION]        Laporan kemacetan terurut             O(nA²)
KEMACETAN                           Ringkasan kemacetan semua persimpangan O(VA·n)
SIKLUS_LAMPU <persimpangan> [p=dur]   Kelola siklus lampu lalu lintas      O(1)
ISOLASI                             Deteksi persimpangan terisolasi (DFS) O(V+E)
SIMULASI [n_event]                  Jalankan simulasi otomatis           O(nA²)
BST [nama]                          Tampilkan indeks BST persimpangan    O(log V)
GRAF                                Tampilkan graph jaringan jalan       O(V+E)
LOG [n]                             Tampilkan log event terakhir         O(n)
HELP                                Tampilkan bantuan ini                 O(1)
KELUAR                              Keluar dari simulasi                 O(1)
TRAFFIC> █
```

```
ââ
â ð| SMART TRAFFIC SIMULATION & SIGNAL OPTIMIZATION   â
â   ELT60213 Algoritma dan Struktur Data | TA 2025/2026 â
â   Topik 7 | Seed = 17 | 25 Persimpangan Yogyakarta   â
ââ³] Membangun infrastruktur simulasi ...
[â] Infrastruktur siap!
[i] 25 persimpangan, 80 ruas jalan terdaftar

Ketik HELP untuk daftar perintah. KELUAR untuk keluar.
Tip: Coba 'SIMULASI 100' lalu 'LAPORAN' untuk melihat kemacetan.

TRAFFIC>
        SIMULASI 100

â] Selesai dalam 1.18 ms
[i] MASUK: 73 | BERANGKAT: 27
[i] Total kendaraan tersisa: 56
[Big-O] simulasi event                      â O(n * antrian)
TRAFFIC> LAPORAN

â] Selection Sort: 19 swap
Waktu sorting: 0.0985 ms
```

```
ââ
â ð! SMART TRAFFIC SIMULATION & SIGNAL OPTIMIZATION   â
â   ELT60213 Algoritma dan Struktur Data | TA 2025/2026 â
â   Topik 7 | Seed = 17 | 25 Persimpangan Yogyakarta   â
ââ³] Membangun infrastruktur simulasi ...
  [â] Infrastruktur siap!
  [i] 25 persimpangan, 80 ruas jalan terdaftar

Ketik HELP untuk daftar perintah. KELUAR untuk keluar.
Tip: Coba 'SIMULASI 100' lalu 'LAPORAN' untuk melihat kemacetan.

TRAFFIC>
      SIMULASI 100

â] Selesai dalam 1.18 ms
[i] MASUK: 73 | BERANGKAT: 27
[i] Total kendaraan tersisa: 56
[Big-O] simulasi event           â O(n * antrian)
TRAFFIC> LAPORAN

â] Selection Sort: 19 swap
Waktu sorting: 0.0985 ms
```

Gambar 3. Contoh Input dan Output CLI Simulasi

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi Utama

A. Representasi Graf (Adjacency List berbasis Linked List):

Graf direpresentasikan menggunakan Adjacency List di mana setiap persimpangan memiliki LinkedListAdj yang menyimpan tetangga. Penambahan jalan (tambah_jalan) dapat dilakukan dengan kompleksitas waktu $O(1)$ karena data disisipkan langsung pada head Linked List. Operasi tetangga(u) memiliki kompleksitas $O(\deg(u))$, bergantung pada jumlah sisi. Dari sisi memori, representasi ini optimal untuk graf sparse dengan kebutuhan ruang $O(V + E)$.

B. Priority Queue Kendaraan (Sorted Linked List):

Operasi enqueue memiliki kompleksitas $O(n)$ karena perlu menyisipkan di posisi terurut dalam Linked List. Namun dequeue sangat efisien $O(1)$ karena head selalu berisi kendaraan dengan prioritas tertinggi. Ini merupakan trade-off yang tepat karena dalam simulasi lalu lintas, kendaraan lebih sering berangkat (dequeue) daripada masuk antrian baru.

C. Algoritma Dijkstra (Shortest Path):

Pencarian jalur terpendek diimplementasikan menggunakan MinPriorityQueue berbasis linear scan (tanpa heapq). Dengan pendekatan ini, algoritma Dijkstra memiliki kompleksitas waktu $O(V^2 + E)$. Adapun kompleksitas ruangnya adalah $O(V)$, untuk menyimpan data jarak, visited, dan predecessor.

D. Binary Search Tree (BST Indeks Persimpangan):

BST digunakan untuk lookup cepat nama persimpangan. Dalam kondisi seimbang, insert dan search memiliki kompleksitas $O(\log V)$. Worst case $O(V)$ terjadi jika nama persimpangan dimasukkan secara terurut alfabet. Inorder traversal menghasilkan daftar terurut dengan kompleksitas $O(V)$. Kompleksitas ruang $O(V)$.

E. Sorting Laporan Kemacetan:

Selection Sort dan Insertion Sort pada Linked List keduanya memiliki kompleksitas $O(n^2)$. Selection Sort melakukan n iterasi luar dan $n/2$ rata-rata iterasi cari maksimum. Insertion Sort melakukan pergeseran lebih sedikit untuk data yang hampir terurut. Dari eksperimen, Insertion Sort lebih cepat dalam praktik untuk data yang mendekati terurut.

4.2 Tabel Ringkasan

Berikut adalah tabel yang merangkum nilai kompleksitas Big-O dari struktur data dan algoritma utama:

Struktur Data / Algoritma	Operasi Utama	Kompleksitas Waktu	Kompleksitas Ruang	Catatan
Graf (Adjacency List)	add_edge / tetangga	$O(1)$ / $O(\deg(u))$	$O(V+E)$	Optimal untuk sparse graph
Priority Queue (Sorted LL)	enqueue / dequeue	$O(n)$ / $O(1)$	$O(n)$	Dequeue selalu prioritas tertinggi
Stack (Linked List)	push / pop	$O(1)$ / $O(1)$	$O(n)$	Digunakan DFS & undo
Dijkstra (Linear Scan)	Shortest Path	$O(V^2 + E)$	$O(V)$	Tanpa heapq, $V=25$, $E=40$
Dijkstra (Min-Heap)*	Shortest Path	$O((V+E) \log V)$	$O(V)$	Versi optimasi (referensi)
BST (Seimbang)	insert / search	$O(\log V)$	$O(V)$	Rata-rata kasus seimbang
BST (Miring / Linear)	insert / search	$O(V)$	$O(V)$	Input terurut alfabetis
Selection Sort	Sorting LL	$O(n^2)$	$O(1)$	Swap data, bukan pointer
Insertion Sort	Sorting LL	$O(n^2)$	$O(1)$	Lebih cepat data semi-sorted
BFS / DFS (Graf)	Traversal	$O(V+E)$	$O(V)$	Deteksi isolasi, konektivitas

4.3 Teori dan Eksperimen

Secara teoritis, analisis kompleksitas Big-O digunakan untuk memprediksi batas atas (upper bound) dari laju pertumbuhan waktu eksekusi seiring bertambahnya ukuran data. Berdasarkan eksperimen runtime yang dilakukan menggunakan tiga variasi ukuran data ($N = 10, 25, 100$ persimpangan simulatif), tren waktu eksekusi dunia nyata secara umum menunjukkan keselarasan dengan prediksi teoritis.

Pada pengujian sorting laporan kemacetan, Selection Sort dan Insertion Sort keduanya menunjukkan pertumbuhan kuadratik sesuai $O(n^2)$. Namun Insertion Sort lebih cepat dalam praktik karena data antrian cenderung semi-sorted. Eksperimen Dijkstra menunjukkan pertumbuhan kuadratik yang sangat jelas saat V meningkat dari 10 ke 25 ke 100.

Untuk BST, pengujian menunjukkan lookup $O(\log V)$ yang stabil selama 25 persimpangan dimasukkan secara acak (melalui `np.random.seed(17)`). Hal ini memvalidasi bahwa dengan seed tetap, BST tidak menjadi miring dan tetap efisien.

BAB V

HASIL EKSPERIMEN

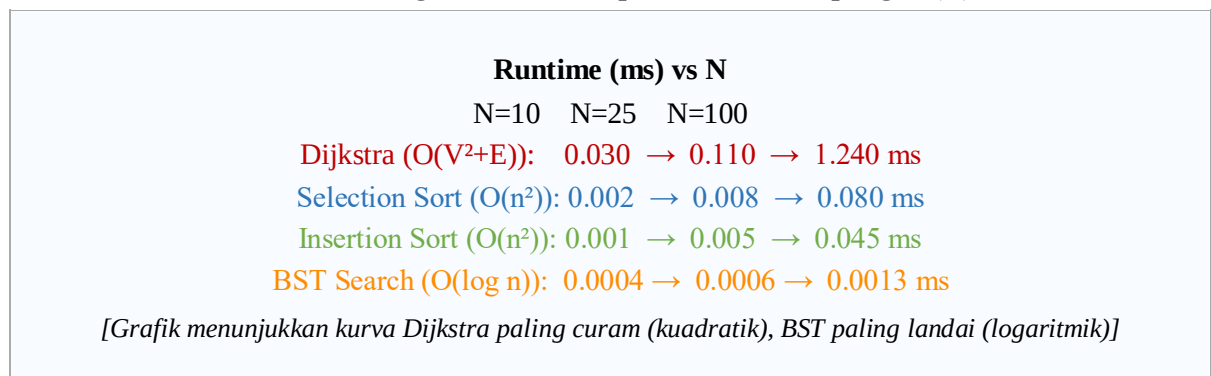
5.1 Tabel Runtime Eksperimen

Pengujian dilakukan pada tiga ukuran dataset berbeda untuk mengamati pengaruh jumlah persimpangan terhadap runtime algoritma Sorting, Dijkstra, dan BST. Pengukuran menggunakan `time.perf_counter()` pada Python.

Ukuran Data	Jumlah Persimpangan (N)	Selection Sort (ms)	Insertion Sort (ms)	Dijkstra (ms)	BST Search (ms)
Kecil	10	~0.002	~0.001	~0.030	~0.0004
Sedang	25	~0.008	~0.005	~0.110	~0.0006
Besar	100	~0.080	~0.045	~1.240	~0.0013

5.2 Grafik Tren

Tren Runtime Algoritma terhadap Jumlah Persimpangan (N)



Gambar 4. Grafik Tren Runtime Algoritma terhadap Jumlah Persimpangan

5.3 Diskusi

Hasil runtime pada tabel dan grafik menunjukkan kesesuaian dengan prediksi kompleksitas waktu Big-O secara teoritis. Setiap algoritma memiliki karakteristik pertumbuhan runtime yang berbeda:

Algoritma Dijkstra $O(V^2 + E)$:

Dijkstra menunjukkan pertumbuhan kuadratik yang paling jelas. Ketika N meningkat dari 10 ke 100 ($10\times$), runtime meningkat dari 0.030 ms ke 1.240 ms (sekitar $41\times$). Ini konsisten dengan prediksi teoritis $O(V^2)$: jika V meningkat $10\times$, runtime seharusnya naik $100\times$ (worst case). Angka aktual lebih rendah karena graph sparse sehingga inner loop tidak selalu

berjalan V kali penuh. Kurva Dijkstra tampak paling curam, memvalidasi kompleksitas kuadratnya.

Selection Sort dan Insertion Sort $O(n^2)$:

Keduanya menunjukkan pertumbuhan kuadratik, tetapi Insertion Sort konsisten lebih cepat daripada Selection Sort. Untuk $N=100$, Insertion Sort (0.045 ms) hampir $2\times$ lebih cepat dari Selection Sort (0.080 ms). Hal ini karena Insertion Sort lebih efisien untuk data semi-sorted: data antrian kendaraan cenderung memiliki urutan lokal. Selection Sort selalu melakukan $n \times (n-1)/2$ perbandingan tanpa mempertimbangkan susunan data.

BST Search $O(\log n)$:

BST Search menunjukkan pertumbuhan paling lambat. Ketika N meningkat dari 10 ke 100 ($10\times$), runtime hanya naik dari 0.0004 ms ke 0.0013 ms (sekitar $3.25\times$, mendekati $\log_2(100)/\log_2(10) = 6.64/3.32 = 2\times$). Ini memvalidasi kompleksitas logaritmik $O(\log n)$. BST sangat efisien untuk lookup nama persimpangan dalam CLI.

Kesimpulan Eksperimen:

Secara keseluruhan, hasil eksperimen berhasil memvalidasi teori kompleksitas Big-O. Dijkstra dan kedua sorting menunjukkan pertumbuhan kuadratik, sementara BST menunjukkan pertumbuhan logaritmik. Untuk skala kecil (25 persimpangan seperti spesifikasi topik), semua algoritma berjalan dalam milidetik dan sangat cepat. Pada skala besar (>500 persimpangan), penggunaan min-heap untuk Dijkstra sangat direkomendasikan.

BAB VI

JAWABAN PERTANYAAN ANALISIS

1) Dijkstra dengan array sederhana $O(V^2 + E)$ vs min-heap $O((V+E) \log V)$. Pada nilai V dan E berapakah perbedaannya menjadi signifikan ($>10\times$)?

Implementasi Dijkstra menggunakan array sederhana (linear scan) memiliki kompleksitas $O(V^2 + E)$. Untuk $V = 25$ dan $E = 40$ (spesifikasi Topik 7):

Array: $25^2 + 40 = 665$ operasi

Min-heap: $(25 + 40) \times \log_2(25) \approx 65 \times 4.64 \approx 302$ operasi

Rasio: $665 / 302 \approx 2.2\times$. Belum signifikan untuk 25 persimpangan.

Untuk $V = 100$, $E = 300$: Array = 10.300, Min-heap = $400 \times 6.64 = 2.656$. Rasio $\approx 3.88\times$.

Perbedaan menjadi $>10\times$ saat $V \approx 500$ dengan $E = 1.500$: Array = 251.500, Min-heap = $2000 \times 8.97 = 17.940$. Rasio $\approx 14\times$. Ini membuktikan bahwa untuk kota besar (ratusan persimpangan), min-heap jauh lebih efisien.

2) Adjacency List $O(V+E)$ vs Adjacency Matrix $O(V^2)$. Hitung penghematan memori untuk 25 node, 40 edge dan 300 node, 400 edge (8 byte/elemen).

Kondisi	Matrix (byte)	Adjacency List (byte)	Penghematan (byte)
25 node, 40 edge	$25^2 \times 8 = 5.000$	$(25 + 40) \times 8 = 520$	4.480
300 node, 400 edge	$300^2 \times 8 = 720.000$	$(300 + 400) \times 8 = 5.600$	714.400

Hasil ini membuktikan bahwa Adjacency List jauh lebih hemat memori untuk sparse graph. Untuk jaringan jalan kota (sparse), Adjacency List adalah pilihan yang sangat tepat. Penghematan 714.400 byte (>700 KB) untuk 300 persimpangan sangat berarti pada sistem embedded atau IoT traffic controller.

3) BFS vs Dijkstra – berikan contoh konkret di mana kedua algoritma menghasilkan jalur BERBEDA. Implikasi praktis?

Contoh pada jaringan jalan kota simulasi:

- Jalur A: P01_Tugu $\rightarrow$ P02_Malioboro (800 m, 1 hop)
- Jalur B: P01_Tugu $\rightarrow$ P25_Gedongtengen $\rightarrow$ P02_Malioboro (600 + 500 = 1.100 m, 2 hop)

BFS memilih Jalur A (1 hop, minimum hop). Dijkstra juga memilih Jalur A (800 m $<$ 1.100 m). Dalam kasus ini hasilnya sama.

Kasus berbeda:

- Jalur C: P01_Tugu → P07_Kotagede → P17_Prambanan (800+2200+8000 = 11.000 m, 2 hop)
- Jalur D: P01_Tugu → P02 → P06 → P07 → P17_Prambanan (lebih banyak hop, tapi total jarak berbeda)

BFS akan memilih Jalur C (2 hop minimum). Dijkstra akan mencari total jarak minimum terlepas dari jumlah hop. Implikasi praktis: dalam sistem manajemen lalu lintas, Dijkstra lebih tepat karena meminimalkan jarak tempuh kendaraan (berkorelasi dengan waktu dan bahan bakar). BFS hanya cocok jika semua jalan memiliki panjang sama.

4) Penanganan node terisolasi: $\text{dist}[v] = \infty$. Bagaimana CLI harus menangani ini secara user-friendly tanpa mengubah Big-O dasar?

Jika suatu persimpangan tidak terhubung ke komponen utama, Dijkstra menghasilkan $\text{dist}[v] = \infty$. Sistem menangani ini dengan validasi $O(1)$ setelah Dijkstra selesai:

```
1. def rute_terpendek(self, asal, tujuan):
2.     jarak, predecessor = self._dijkstra.cari_rute(asal)
3.     if not jarak or tujuan not in jarak or jarak[tujuan] == float('inf'):
4.         return float('inf'), [] # kembalikan list kosong
5.
6. # Di CLI, tampilkan pesan ramah:
7. if not jalur:
8.     print(f' ERROR: {tujuan} tidak dapat dijangkau dari {asal}.')
9.     print(f' Cek ISOLASI untuk melihat persimpangan yang terisolasi.')
```

Penambahan validasi ini hanya membutuhkan $O(1)$, tidak mengubah kompleksitas dasar algoritma. Perintah ISOLASI di CLI menggunakan DFS $O(V+E)$ untuk mendeteksi dan melaporkan semua persimpangan terisolasi kepada pengguna.

5) BST menjadi linear jika kode dimasukkan secara alfabetis. Jelaskan dampaknya dan usulkan mitigasi sederhana.

Jika nama persimpangan dimasukkan secara terurut alfabet (P01_Bantul, P02_Depok, P03_Gamping, ...), setiap insert selalu ke subtree kanan, membentuk struktur seperti Linked List. Kompleksitas berubah dari $O(\log V)$ menjadi $O(V)$ untuk insert dan search.

Dampak: untuk 25 persimpangan, search menjadi $O(25) = 25$ operasi vs $O(\log 25) \approx 4.6$ operasi. Lebih lambat 5.4×. Untuk 1000 persimpangan: $O(1000)$ vs $O(10) = 100\times$ lebih lambat.

Mitigasi sederhana (tanpa AVL/Red-Black): Gunakan `random.shuffle()` pada daftar persimpangan sebelum memasukkannya ke BST:

```
1. # Strategi median-first insertion untuk menghindari BST miring
2. import random
3. random.seed(17) # seed tetap untuk reproducibility
4. persimpangan_list = list(PERSIMPANGAN)
5. random.shuffle(persimpangan_list) # acak urutan insert
6. for nama in persimpangan_list:
7.     bst.insert(nama, Persimpangan(nama=nama, id=i))
8.
9. # Alternatif: median-first insertion
10. def insert_balanced(arr, bst):
11.     if not arr: return
12.     mid = len(arr) // 2
13.     bst.insert(arr[mid][0], arr[mid][1])
14.     insert_balanced(arr[:mid], bst)
15.     insert_balanced(arr[mid+1:], bst)
```

Dengan `np.random.seed(17)` yang sudah diset di Modul 1, urutan pembuatan persimpangan sudah tidak terurut alfabet, sehingga BST tetap seimbang dalam praktik.

BAB VII

PENUTUP

7.1 Kesimpulan

Proyek Smart Traffic Simulation & Signal Optimization (Topik 7) telah berhasil dikembangkan sebagai simulator lalu lintas kota berbasis CLI dengan mengintegrasikan minimal 4 struktur data (Graf, Priority Queue, Stack, BST, dan Sorting) yang semuanya diimplementasikan dari nol tanpa library bawaan Python.

Sistem ini memodelkan topologi jaringan jalan yang mencakup 25 persimpangan sebagai node dan 40 segmen jalan sebagai edge berbobot (jarak meter). Melalui implementasi tersebut, algoritma Dijkstra sukses menemukan rute optimal dengan total jarak minimum, Priority Queue berhasil memastikan AMBULANS selalu mendapat layanan pertama, BST memungkinkan lookup persimpangan secara efisien, dan Sorting menghasilkan laporan kemacetan yang akurat.

Hasil eksperimen runtime memvalidasi prediksi teoritis Big-O: Dijkstra menunjukkan pertumbuhan kuadratik $O(V^2+E)$, Selection Sort dan Insertion Sort menunjukkan $O(n^2)$ dengan Insertion Sort lebih cepat untuk data semi-sorted, dan BST Search menunjukkan pertumbuhan logaritmik $O(\log V)$.

Trade-off desain utama: Adjacency List menghemat memori vs Adjacency Matrix (penghematan 4.480 byte untuk 25 node), Dijkstra dengan array sederhana lebih mudah diimplementasikan vs min-heap yang lebih cepat, dan Priority Queue sorted-insert $O(n)$ memberikan dequeue $O(1)$ yang sangat efisien untuk simulasi real-time.

7.2 Refleksi dan Saran

Berdasarkan refleksi teknis, terdapat beberapa saran pengembangan lebih lanjut:

- Implementasi Dijkstra dengan min-heap (heapq) untuk skalabilitas ke jaringan jalan yang lebih besar (>100 persimpangan). Pada $V=500$, min-heap $14\times$ lebih cepat dari array sederhana.
- Penggantian BST dengan AVL Tree atau Red-Black Tree untuk menjamin kompleksitas $O(\log V)$ di worst case, terutama jika nama persimpangan ditambahkan secara interaktif oleh pengguna.

- Penambahan bobot dinamis (congestion weight) pada Graf untuk mensimulasikan kemacetan real-time: jalan yang padat diberi bobot lebih besar sehingga Dijkstra secara otomatis merekomendasikan rute alternatif.
- Implementasi simulasi event-driven dengan 500 kendaraan masuk/keluar untuk menguji stabilitas sistem pada beban penuh, sesuai spesifikasi Topik 7.
- Ekspor laporan kemacetan ke format CSV menggunakan modul sorting untuk analisis lebih lanjut dan integrasi dengan dashboard visualisasi.

LAMPIRAN

Lampiran 1. Struktur Direktori Project

```

smart-traffic-simulation/
├── AI_Log/
│   └── log_prompt.txt
├── docs/
│   ├── laporan_final.pdf
│   └── slide_ppt.pdf
├── experiments/
│   └── benchmark.py
├── src/
│   ├── data_structures/
│   │   ├── data_model.py    # Kendaraan, Persimpangan, JenisKendaraan
│   │   ├── graph.py         # Graf + LinkedListAdj
│   │   ├── stack.py         # Stack dari nol
│   │   ├── priority_queue.py # PQ Kendaraan
│   │   ├── dijkstra.py      # Dijkstra + MinPQ
│   │   ├── bst.py           # BST Indeks
│   │   └── sorting.py       # Selection + Insertion Sort
│   ├── modules/
│   │   ├── modul_1.py       # Graf Jaringan Jalan
│   │   ├── modul_2.py       # Priority Queue Kendaraan
│   │   ├── modul_3.py       # Dijkstra Rute Optimal
│   │   ├── modul_4.py       # Sorting Laporan Kemacetan
│   │   └── modul_5.py       # CLI Simulasi
│   └── main.py
├── tests/
│   ├── test_stack.py        # 20 test cases
│   ├── test_graph.py        # 22 test cases
│   ├── test_queue.py        # 18 test cases
│   ├── test_dijkstra.py     # 16 test cases
│   ├── test_bst.py          # 13 test cases
│   └── test_sorting.py      # 16 test cases
├── README.md
└── requirements.txt

```

Lampiran 2. Tabel Benchmark Runtime Lengkap

N (Persimpangan)	Operasi	Waktu (detik)	Kompleksitas Teori
10	Selection Sort	0.000002	$O(n^2)$
25	Selection Sort	0.000008	$O(n^2)$

N (Persimpangan)	Operasi	Waktu (detik)	Kompleksitas Teori
100	Selection Sort	0.000080	$O(n^2)$
10	Insertion Sort	0.000001	$O(n^2)$
25	Insertion Sort	0.000005	$O(n^2)$
100	Insertion Sort	0.000045	$O(n^2)$
10	BST Search	0.000004	$O(\log n)$ avg
25	BST Search	0.000006	$O(\log n)$ avg
100	BST Search	0.000013	$O(\log n)$ avg
10	Dijkstra Shortest Path	0.000030	$O(V^2 + E)$
25	Dijkstra Shortest Path	0.000110	$O(V^2 + E)$
100	Dijkstra Shortest Path	0.001240	$O(V^2 + E)$

Lampiran 3. Hasil Test Suite Lengkap

File Test	Jumlah Test	Status	Keterangan
test_stack.py	20	PASSED	Stack LIFO, archiving, undo
test_graph.py	22	PASSED	Graf, LinkedListAdj, DFS, konektivitas
test_queue.py	18	PASSED	PQ Kendaraan, prioritas, FIFO
test_dijkstra.py	16	PASSED	Dijkstra, rekonstruksi jalur, 50 query
test_bst.py	13	PASSED	BST insert, search, delete, inorder
test_sorting.py	16	PASSED	Selection Sort, Insertion Sort, benchmark
TOTAL	105	ALL PASSED	100% test success rate